



Today's agenda

- ↳ find all anagrams
- ↳ minimum window substring
- ↳ Anagram Palindrome
- ↳ more problems

→ Anagram is fancy word for Permutation.

S: "abc"

P: "cba" ✓

S: "aabcd"

P: "bacaad" ✓

S: "abcd"

P: "abd" ✗✗



Q) All Anagrams

↳ Given 2 Strings S and P , return an array of all start indices of P 's anagram in S .

Ex: $S = \text{"cbaebabacd"}$
 $P = \text{"abc"}$

//ideal

↳ compare every starting points with P .

ans = []

$S = \text{"cbaebabacd"}$
 $P = \text{"abc"}$

hmap

a-1
e-1
b-1

matchcount = 0/2

hmap

a-1
b-1
c-1

T.C: $O(S \times P)$

S.C: $O(P)$



11 idea 2

S: "c b a e b a b a c d"

P: "a b c"

(0, 1, 2)

(1, 2, 3)

(2, 3, 4)

hms

b-x2x2
a-x21
e-x0

hmp

a-1
b-1
c-1

match count = 0x2x2

ans = 0

	acquire	mc increased?	release	mc dec?	match count
iter 0:	e	x	c	✓	2
iter 1:	b	x	b	x	2
iter 2:	a	x	a	x	2
iter 3:	b	x	e	x	2



// Pseudo code

```
List<Integer> findAnagrams (String s, String p) {  
    if (p.length() > s.length()) { return new ArrayList(); }
```

```
    HashMap<Character, Integer> hms, hmp;
```

```
    for (int i=0; i<p.length(); i++) {  
        hmp.put (p[i], hmp.getOrDefault(p[i], 0)+1);  
    }
```

T.C: $O(p + p + s \cdot p) = O(p \cdot s)$ int matchcount=0;

S.C: $O(s)$

```
List<Integer> ans;
```

```
    for (int i=0; i<p.length; i++) {  
        hms.put (s[i], hms.getOrDefault(s[i], 0)+1)  
        if (hms.get(s[i]) <= hmp.getOrDefault(s[i], 0)) {  
            matchcount++;  
        }  
    }  
}
```

```
    if (matchcount == p.length()) { ans.add (0); }
```

```
    int sp=0;
```

```
    int ep = p.length();
```

```
    while (ep < s.length()) {
```

```
        hms.put (s[ep], hms.getOrDefault(s[ep], 0)+1)
```

```
        if (hms.get(s[ep]) <= hmp.getOrDefault(s[ep], 0)) {  
            matchcount++;  
        }
```

```
    }
```



```
hms.put (s[sp], hms.get(s[sp]) - 1);
```

```
if (hms.get(s[sp]) < hmb.getOrDefault(s[sp], 0))  
    matchcount--;
```

```
}
```

```
sp++;
```

```
ep++;
```

```
if (matchcount == p.length()) { ans.add(sp); }
```

```
}
```

```
return ans;
```

```
}
```



AlgoPrep

^{sp} ^{ep}
 S: "c b a e b a b a c d"
 P: "a b c"

hms

c - 20
 b - 1
 a - 1
 e - 1

hmp

a - 1
 b - 1
 c - 1



matchcount = 02232

ans = 10

```

int matchcount = 0;
List<Integer> ans;
for (int i = 0; i < P.length; i++) {
    hms.put(S[i], hms.getOrDefault(S[i], 0) + 1);
    if (hms.get(S[i]) <= hmp.getOrDefault(S[i], 0)) {
        matchcount++;
    }
}
if (matchcount == P.length()) { ans.add(0); }
  
```

```

int sp = 0;
int ep = P.length();
while (ep < S.length()) {
    hms.put(S[ep], hms.getOrDefault(S[ep], 0) + 1);
    if (hms.get(S[ep]) <= hmp.getOrDefault(S[ep], 0)) {
        matchcount++;
    }
}

hms.put(S[sp], hms.get(S[sp]) - 1);
if (hms.get(S[sp]) < hmp.getOrDefault(S[sp], 0)) {
    matchcount--;
}
sp++;
ep++;

if (matchcount == P.length()) { ans.add(sp); }
  
```



Q) Anagram Palindrome

↳ Given $S[n]$, check whether any anagram of the string is palindrome or not.

Ex: ALGOPREP → false

ALGOPOLPL → True
↓
L G O P A P O L L

Idea 1

↳ Create all the Anagrams and check for Palindrome.

Idea 2

↳ All the characters should occur even no. of times.

↳ If one character is occurs odd no. of times, still works.

Ex: a a a a b b d d d

a a b d d d b a a

Break till 9:40 PM

leetcode 76



Q) Minimum window substring

↳ Given two string $S[m]$ and $t[n]$, return the minimum window substring of S such that every character in t is included in the window.

Ex: $S: ADOBE CODE BANC$ $T: ABC$

Idea 1

↳ Generate all the substrings and check if characters of T are present or not, pick the smallest out of them.

T.C: $O(S^2 * T)$

S.C: $O(T)$

Idea 2

$S: ADOBE CODE BANC$

$T: ABC$

len = 6 5 4



//Pseudo Code

```
String minWindowString (String s, String t) {  
    if (t.length() > s.length()) return "...";  
}
```

```
HashMap < Character, Integer > smap, tmap;  
int matchCount = 0;
```

```
for (int i = 0; i < t.length; i++) {  
    tmap.put (t.charAt(i), tmap.getOrDefault(t.charAt(i), 0) + 1);  
}
```

```
int ansLen = t.length();  
int start = 0;  
int end = 0;
```

```
while (end < s.length()) {
```

```
    if (matchCount == t.length()) {
```

```
        if (end - start < ansLen) {
```

```
            ansLen = end - start;  
            ansStart = start;  
            ansEnd = end - 1;
```

```
        }
```

```
        smap.put (s.charAt(start), smap.get(s.charAt(start)) + 1);
```

```
        if (smap.get(s.charAt(start)) < tmap.getOrDefault(s.charAt(start), 0)) {
```

```
            matchCount++;
```

```
        }
```

```
        start++;
```

```
    }
```

T.C: $O(s)$

S.C: $O(s)$

else {

Smal.put (s[sp], Smal.getOrDefault(s[sp], 0) + 1);

if (Smal.get(s[sp]) <= tmal.getOrDefault(s[sp], 0)) {

matchcount ++;

}

ep ++;

}

}

while (match.count == s.length()) {

if (ep - sp < anslen) {

anslen = ep - sp;

}

Smal.put (s[sp], Smal.get(s[sp]) - 1);

if (Smal.get(s[sp]) < tmal.getOrDefault(s[sp], 0)) {

matchcount --;

}

sp ++;

}

return s.substring (anslen, anslen + 1);

}

Do your



map

A-1
B-1
C-1

0 1 2 3 4 5 6 7 8 9 10 11 12
S: A D O B E C O D E B A N C
T: A B C

map

A-1 e-2
D-2 C-1 N-1
O-1 O-1
B-1 D-1

int anslen = 0;
int sp = 0;
int ep = 0;

while (ep < s.length())

if (matchcount == t.length())

if (ep - sp < anslen)

anslen = ep - sp;

map.put(s[sp], map.get(s[sp]) + 1);

if (map.get(s[sp]) < map.getOrDefault(s[sp], 0))

matchcount--;

sp++;

else

map.put(s[ep], map.getOrDefault(s[ep], 0) + 1);

if (map.get(s[ep]) < map.getOrDefault(s[ep], 0))

matchcount++;

ep++;

anslen = 6

matchcount = 2 2 2 3